

Contents

Editorial	2
OUTILS DE COLLABORATION WEB : Étude exploratoire	3
1. Objet	3
2. Démarche exploratoire	3
3. Options du marché	3
4. Options architecturales	5
5. Option retenue	8
Quantum Support Vector Classification with the Titanic Dataset	9
I. Introduction	9
II. Choix des données	11
III. Choix de l'algorithme implémenté en prototype	14
IV. Les méthodes utilisées dans le projet Titanic	16
V. Conclusion	18
Références	18
Jumpstart Reinforcement Learning with PPO	20
1. Introduction	20
2. Méthodologie	22
3. Résultats	23
4. Discussion	24
5. Conclusion	25
Références	26
Mentions Légales	27

Editorial

Grégoire Cattan

France, 2024

OUTILS DE COLLABORATION WEB : Étude exploratoire

Marc Lefrançois

Le-Point-Technique, 06/2024

abstract: Étude exploratoire des solutions de collaboration Web (visioconférence et partage de fichiers) avec analyse du marché, comparaison architecturale et recommandations pour une entreprise.

keywords: visioconférence, partage de fichiers, architecture logicielle, NextCloud, Jitsi

1. Objet

Le présent document concerne l'étude exploratoire dans le cadre d'une solution Web de collaboration, réunion et présentation pour une entreprise. Son objectif est de présenter les différentes options d'architecture et d'étudier les différentes solutions pour les composants de visioconférences et de partage de fichier.

2. Démarche exploratoire

La démarche de l'étude exploratoire a été réalisée selon trois étapes :

1. Étude des options du marché
 2. Étude des options architecturales
 3. Sélection de la solution retenue
-

3. Options du marché

3.1. Outil de visioconférence

3.1.1. Cisco WebEx Meetings (Webex Suite) **Description :** solution clé en main (appels audio/vidéo, visioconférence, messagerie instantanée, sondages, événements).

Avantages : support, compatibilité multiplateforme, chiffrement de bout en bout, conformité RGPD, API d'extension, choix de la région de stockage.

Inconvénients : propriétaire, données stockées chez un tiers, coût mensuel (30–50€/licence), pas de support Linux.

Conclusion : 90 % des besoins couverts mais non retenue (coût + absence d'intégration).

3.1.2. Zoom **Description** : messagerie et visioconférence (appels, messagerie, calendrier, événements).

Avantages : support, mise en place rapide, API, chiffrement, conformité RGPD.

Inconvénients : propriétaire, données chez un tiers, coût (17€/licence ou plus), pas de Linux, suspicion sur la collecte des données.

Conclusion : 85 % des besoins couverts mais non retenue (coût + confidentialité).

3.1.3. Microsoft Teams **Description** : communication collaborative intégrée à Microsoft 365.

Avantages : support, interconnectivité dans l'écosystème Microsoft, chiffrement, conformité RGPD.

Inconvénients : propriétaire, dépendance à Microsoft 365, coût (~12€/licence), pas d'enregistrement natif, pas d'API d'extension.

Conclusion : 80 % des besoins couverts mais non retenue (coût + verrou Microsoft).

3.1.4. Solution applicative Google **Description** : Google Meet (visioconférence) + Google Chat (messagerie).

Avantages : solution clé en main, chiffrement, conformité RGPD.

Inconvénients : propriétaire, collecte de données possible, coût (~12€/licence), dépendance au cloud Google.

Conclusion : 95 % des besoins couverts mais non retenue (coût + intégration limitée).

3.1.5. Jitsi **Description** : application libre (OpenSource) pouvant fonctionner en JaaS.

Avantages : marque blanche, coût maîtrisé, données internes, extensible via API et code source, adoption étatique française, pas de limite logicielle de participants.

Inconvénients : nécessite des moyens d'intégration et de maintenance supplémentaires.

Conclusion : répond à 100 % des besoins, d'où le statut **retenue**.

3.1.6. NextCloud Talk **Description** : extension NextCloud pour la collaboration (appels, visioconférence, messagerie).

Avantages : OpenSource, marque blanche, maîtrise des coûts, extensible, données internes, adoption ministérielle, pas de limitation logicielle.

Inconvénients : besoin de moyens pour intégration et maintien.

Conclusion : répond à 98 % des besoins, d'où le statut **retenue**.

3.2. Outil de partage de fichiers

3.2.1. Google Drive **Avantages** : intégration avec de nombreux outils, chiffrement, conformité RGPD, antivirus intégré.

Inconvénients : propriétaire, données chez un tiers, coût mensuel, suspicion sur la collecte de données.

Conclusion : non retenue.

3.2.2. NextCloud Files **Avantages** : OpenSource, marque blanche, extensible, chiffrement, conformité RGPD, adoption ministérielle.

Inconvénients : nécessite des moyens d'intégration et de maintenance.

Conclusion : répond à 98 % des besoins, d'où le statut **retenue**.

3.2.3. SharePoint **Avantages** : intégration Microsoft, extensible, déploiement interne possible.

Inconvénients : propriétaire, dépendance Microsoft, offre trop large, coût mensuel (~28€/licence), pas Linux.

Conclusion : non retenue.

3.2.4. Liferay **Avantages** : OpenSource, extensible en JEE, coûts maîtrisés.

Inconvénients : nécessite des compétences spécifiques et un temps d'implémentation élevé.

Conclusion : non retenue.

3.3. Conclusion

Solutions retenues :

- **Visioconférence** : Jitsi ou NextCloud Talk
 - **Partage de fichiers** : NextCloud Files
-

4. Options architecturales

4.1. Critères architecturaux

- **Évolutivité** : ajout efficace de nouvelles solutions.
- **Simplicité** : bonne granularité, documentation à jour, réutilisabilité.
- **Maintenabilité** : support du MCO, outils d'investigation, évolutivité.
- **Compatibilité** : avec les plateformes matérielles et logicielles.
- **Interconnectivité** : standards d'interfaçage (ETL, Web services).

4.2. Options architecturales

4.2.1. Typologie architecturale (Voir tableau dans Table 1 ci-dessous)

ARCHITECTURE	EXEMPLE D'APPLICATION
Client-serveur	
Pilotée par événements	Micro-blogging, automatisation d'usine
Orientée services	Suivi de colis, validation bancaire
Modulaire	Progiciel avec extensions VBA
En couches	Pile TCP/IP, DAO
Centrée sur les données	CRM

4.2.2. Architecture client-serveur Répartit les tâches entre les fournisseurs de ressources (serveurs) et les consommateurs de ces ressources (clients), qui y accèdent par le biais d'ordinateurs, tablettes ou smartphones. Ce type d'architecture centralise et facilite l'accès aux ressources, tout en séparant les aspects matériel, logiciel et fonctionnel, ce qui en facilite la maintenance.

4.2.3. Architecture pilotée par les événements S'organise autour d'un bus d'événements informationnels, avec en entrée les différents types d'événements survenant (les déclencheurs) et en sortie les canaux correspondant à la nature de l'événement. Elle permet de répondre à des situations non prédictives, en adoptant un comportement spécifique à chaque événement.

4.2.4. Architecture orientée services Centralise des services Web internes (site Web, Extranet, Intranet) ou externes (fournisseurs indépendants tels que des transporteurs ou des banques), afin de les mettre à disposition des clients en toute transparence et de faciliter leur expérience utilisateur.

4.2.5. Architecture modulaire S'appuie sur un dispositif permettant d'ajouter des fonctionnalités non prévues initialement, en étendant les fonctionnalités de base afin de les spécialiser selon les besoins des utilisateurs, tout en facilitant la maintenance et la réutilisabilité.

4.2.6. Architecture en couches S'articule autour d'une pile organisée par thèmes fonctionnels, afin de traiter indépendamment chaque aspect dans le cadre de processus verticaux, ce qui facilite également la maintenance.

4.2.7. Architecture centrée sur les données Permet l'implémentation de stratégies (par exemple commerciales) en gérant les règles de gestion dans des tableaux plutôt que dans le code, ce qui facilite la maintenabilité : la logique du système est basée sur ses données plutôt que sur des règles fixées intrinsèquement par le code.

4.3. Architecture cible de visioconférence

4.3.1. Architectures non retenues

- **Client-serveur** : il ne s'agit pas de mise à disposition de ressources centralisées, mais de diffusion d'informations (voix, vidéos, texte, fichiers).
- **Orientée services** : il ne s'agit pas de mise à disposition de services internes ou externes, mais de diffusion d'informations.
- **Modulaire** : le système actuel ne dispose d'aucun dispositif visant à étendre ses fonctionnalités ; ce n'est donc pas, à proprement parler, une architecture modulaire.
- **En couches** : le besoin de visioconférence requiert une architecture plutôt horizontale que verticale (diffuser de l'information d'un point vers un ou plusieurs autres), au moins pour l'aspect logiciel — l'architecture technique (protocole TCP/IP) pourrait, elle, s'en accommoder.
- **Centrée sur les données** : le besoin ne comporte pas d'implémentation de stratégies métier visant à orienter les utilisateurs.

4.3.2. Architecture retenue Il s'agit de l'architecture pilotée par événements. De par sa composition (bus, événements, canaux de diffusion) et son objectif de répondre à la survenue d'événements non prédictifs par l'adoption d'un comportement informatif spécifique à chacun d'eux, elle est adaptée à la diffusion d'informations dédiées à des groupes d'utilisateurs identifiés, et remplit donc les fonctions attendues de l'outil de visioconférence.

4.4. Architecture cible de partage de fichiers

4.4.1. Architectures non retenues

- **Pilotée par événements** : il ne s'agit pas d'une situation non prédictive à laquelle répondre par la diffusion d'informations à un groupe d'utilisateurs — c'est l'inverse : l'utilisateur accède, selon son niveau d'autorisation, à un ensemble de ressources (ajout, modification, suppression) sans dépendre du déclenchement d'actions par d'autres.
- **Orientée services** : il s'agit ici de centralisation de ressources documentaires, pas de mise à disposition de services internes ou externes.
- **Modulaire** : de même qu'en 4.3.1, le système actuel n'est pas, à proprement parler, une architecture modulaire.
- **En couches** : le besoin de partage de fichiers requiert une architecture visant à centraliser et mettre à disposition des ressources aux utilisateurs,

plutôt qu'une organisation verticale par couches — au moins pour l'aspect logiciel.

- **Centrée sur les données** : le besoin ne comporte pas d'implémentation de stratégies métier visant à orienter les utilisateurs.

4.4.2. Architecture retenue Il s'agit de l'architecture client-serveur. Son objectif — centraliser et faciliter l'accès à des ressources pour des clients (consommateurs) — correspond exactement au besoin de partage de documents énoncé en section 3.

5. Option retenue

5.1. Comparatif interne vs externe

5.1.1. Solution interne Avantages : sur-mesure, maîtrise des coûts, indépendance fournisseur.

Inconvénients : coûts humains/financiers élevés, délais longs, risques sécurité.

5.1.2. Solution externe Avantages : clé en main, support, délais réduits.

Inconvénients : dépendance fournisseur, coût mensuel possible.

5.1.3. Conclusion Choix : **solution externe**.

5.2. Propriétaire vs OpenSource

Conclusion : OpenSource préférable (indépendance, personnalisation, coûts).

5.3. Choix final

Solution retenue : **NextCloud** (NextCloud Talk + NextCloud Files).

Quantum Support Vector Classification with the Titanic Dataset

Adrien Veres

Le-Point-Technique, 06/2024

abstract: Ce travail explore l'apport de l'informatique quantique au machine learning en comparant un classifieur quantique, le QuanticSVM de la librairie pyRiemann-qiskit, à des méthodes classiques (régression linéaire, SVC, SVC à noyau RBF) sur le jeu de données Titanic. L'objectif est d'évaluer si un noyau quantique peut améliorer la prédiction du taux de survie des passagers par rapport à la méthode actuellement utilisée en production chez un client.

keywords: quantum machine learning, QuanticSVM, pyRiemann-qiskit, Qiskit, Titanic, SVM

I. Introduction

A. Un domaine en pleine évolution

La data science, ou « science des données », regroupe plusieurs disciplines : les mathématiques, les statistiques, la visualisation, le machine learning, etc. (*Figure 1*).

Le machine learning, ou « apprentissage machine », est la discipline qui permet à l'ordinateur d'apprendre à partir d'un jeu de données, afin de faire des prédictions, établir des tendances ou même prendre des décisions en fonction de ces données.

La data science est en plein essor : de plus en plus d'entreprises y font appel, de la finance à la santé. Elle aide les gouvernements à détecter les fraudes fiscales, à gérer les risques dans les fonds d'investissement, à trouver les meilleures chances de réussite pour soigner les malades à l'hôpital, etc. On peut la mettre en place dans presque tous les secteurs, même le BTP par exemple pour prédire la consommation d'énergie à l'avance.

B. Objectif du projet

L'objectif du projet est d'améliorer la méthode actuellement utilisée en production chez un client pour prédire le taux de survie des passagers du Titanic. Pour ce faire, nous allons explorer de nouvelles avancées, et le choix du nouvel algorithme à tester s'est porté sur le QuanticSVM.

Le SVM (Support Vector Machine) est une technique de classification largement utilisée. Dans ce travail, on s'intéresse à une version quantique du SVM. La littérature scientifique a démontré que dans certaines circonstances, les algorithmes quantiques permettent d'améliorer la performance de la classification

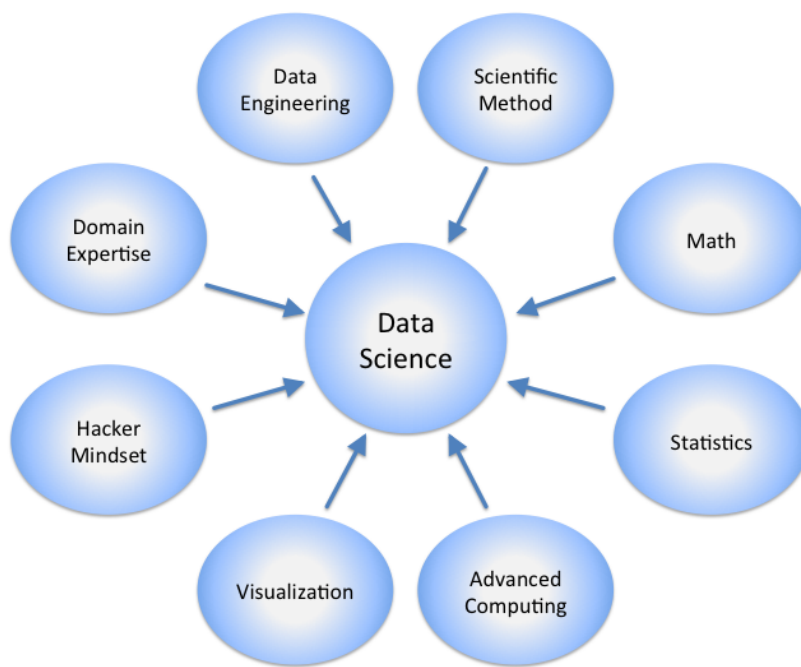


Figure 1: Les disciplines de la data science

(Liu, Arunachalam, and Temme 2021). Cependant, dans la plupart des cas, les évaluations ont été réalisées avec des données artificielles, pas réelles.

Nous allons donc essayer d’appliquer ces méthodes à un vrai jeu de données, celui du Titanic, en comparant la performance du SVR avec un noyau quantique et un noyau RBF, ainsi qu’avec une régression linéaire simple. Nous nous inspirons du travail réalisé par (Andreev et al. 2023), qui présente une classification quantique de vraies données (dans cet article, il s’agit de données encéphalographiques).

C. Notes techniques

Dans ce travail, on utilise la librairie `pyRiemann-qiskit`, qui est une sur-couche autour de la librairie `Qiskit`, une bibliothèque quantique développée par IBM. Elle a été conçue pour être aussi proche que possible des interfaces de `scikit-learn`, dans l’objectif de réduire l’écart entre les algorithmes classiques et les algorithmes quantiques.

L’intérêt de la sur-couche `pyRiemann-qiskit` est de permettre de configurer plus facilement le classifieur. Elle permet aussi de classer non seulement des vecteurs, mais aussi des matrices, grâce à une méthode appelée la « géométrie riemannienne » (Congedo, Barachant, and Bhatia 2017).

II. Choix des données

1. Explication du choix du projet utilisé comme référence

Nous avons choisi, avec mon mentor, de reprendre notre projet Titanic dans l’objectif de potentiellement améliorer notre score SVC à noyau RBF avec le `QuanticSVM` de `pyRiemann`.

2. Description du jeu de données sélectionné

Le projet Titanic est à la base une compétition Kaggle, dont l’objectif est de prédire le taux de survie des passagers du naufrage à partir de plusieurs variables. Nous avons calculé l’accuracy avec la régression linéaire et le SVC, puis le score du SVC à noyau RBF.

Un premier examen des taux de survie par sexe, âge, tarif, port d’embarquement et classe est résumé en *Figure 2*.

On notera en particulier, sur la répartition par classe (*Figure 3*), que le taux de mortalité est beaucoup plus élevé en classe 3, qu’il y a presque autant de décès que de survivants en classe 2, et un taux de survie beaucoup plus élevé pour la classe 1.

Détails des taux de survies :

Taux de survie par sex :

1. Femmes : 74.2%
2. Hommes: 18.8%

Taux de survie par âge :

- (0, 18] 49.31%
- (18, 25] 33.14%
- (25, 40] 37.16%
- (40, 60] 39.28%
- (60, 100] 22.72%

Taux de survie par tarif :

- (0, 10] 20.56%
- (10, 25] 42.08%
- (25, 50] 41.95%
- (50, 100] 65.42%
- (100, 1000] 73.58%

Taux de survie par port d'embarquement :

1. Cherbourg 55.35%
2. Queenstown 38.96%
3. Southampton 33.69%

Taux de survie par classe :

1. 1 62.96%
2. 2 47.28%
3. 3 24.23%

Figure 2: Détails des taux de survie

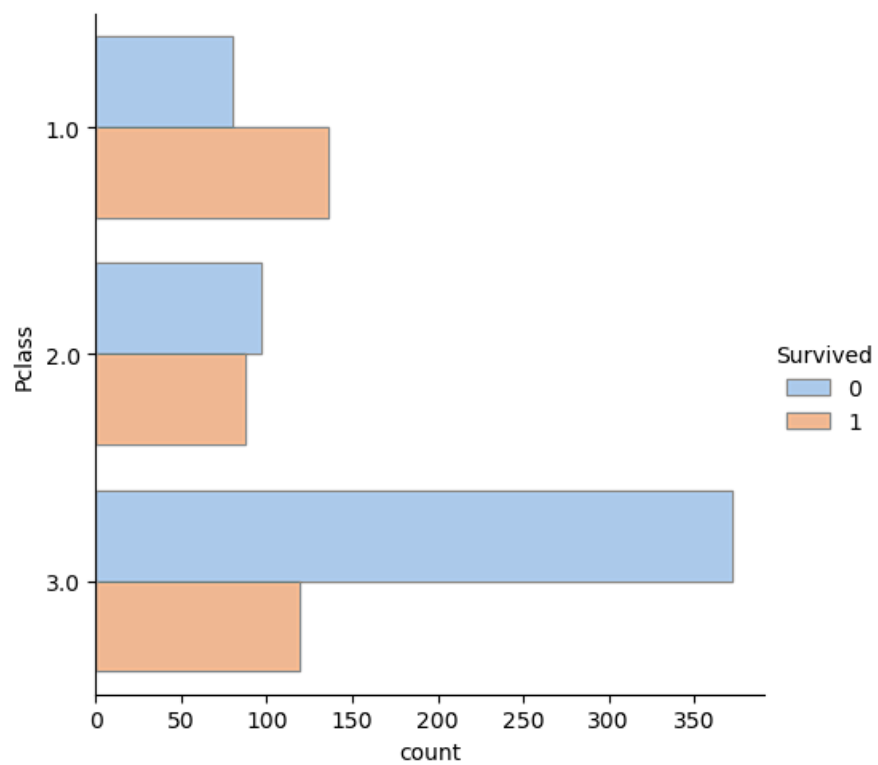


Figure 3: Taux de survie par classe

Concernant le port d'embarquement (*Figure 4*), Southampton a eu le taux de décès le plus élevé et Cherbourg le taux de décès le plus bas ; Queenstown a eu un taux de décès élevé par rapport au nombre de passagers qui y ont embarqué.

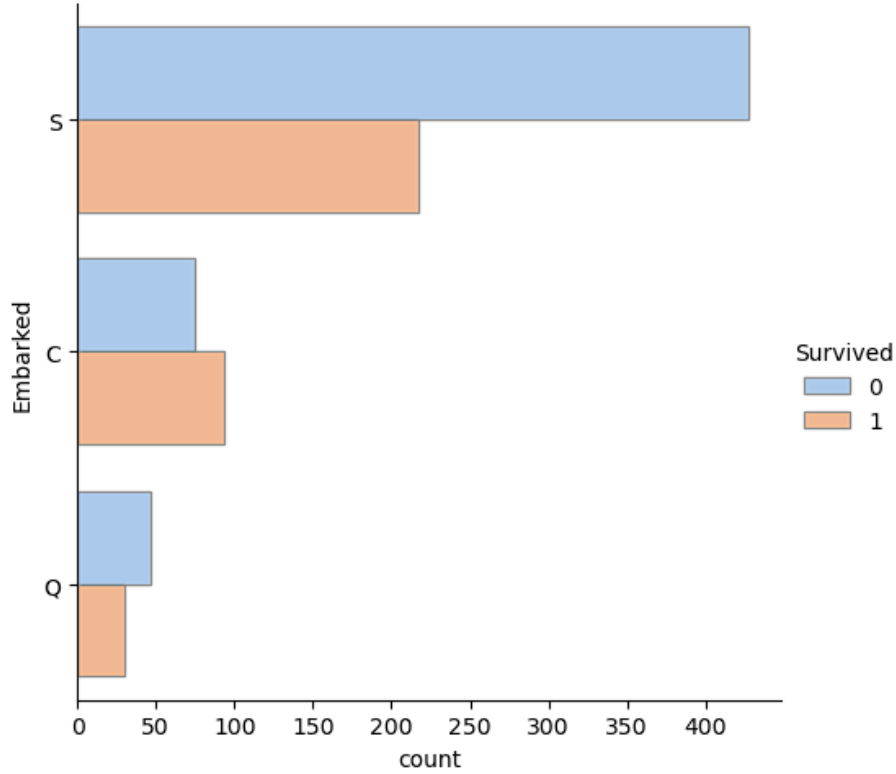


Figure 4: Taux de survie par port d'embarquement

Enfin, concernant le genre (*Figure 5*), on constate que le taux de décès est significativement plus élevé pour les hommes que pour les femmes.

III. Choix de l'algorithme implémenté en prototype

A. Recherche d'un article de recherche récent

Le choix s'est porté sur les travaux de (Andreev et al. 2023) et de (Havlíček et al. 2019), qui traitent respectivement de pyRiemann-qiskit et de l'apprentissage supervisé avec des espaces de caractéristiques quantiques.

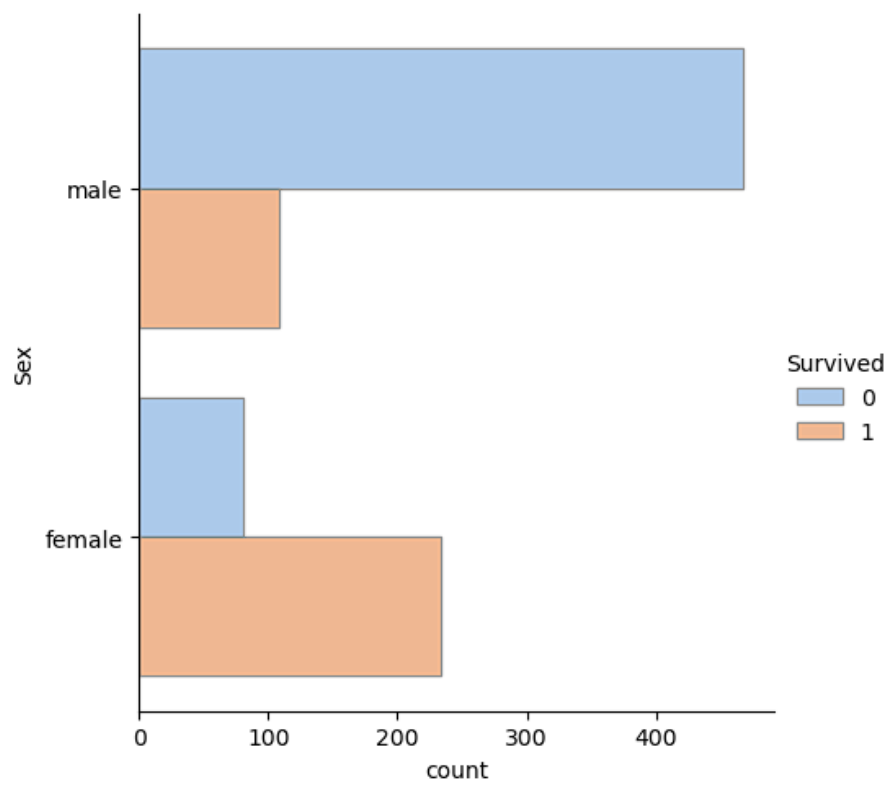


Figure 5: Taux de survie par genre

B. Description de l’algorithme sélectionné et de ses techniques

L’informatique quantique est une technologie prometteuse pour le machine learning, qui peut être moins coûteuse et potentiellement donner de meilleurs résultats. Nous avons décidé d’utiliser Qiskit, une bibliothèque d’informatique quantique développée par IBM, et pyRiemann, qui est un cadre qui aide à analyser les signaux du cerveau en se basant sur la théorie des espaces de Riemann pour extraire des caractéristiques discriminantes à partir de ces signaux et les utiliser dans des tâches de classification.

La fusion de ces deux bibliothèques a permis la création d’un processus standardisé appelé `QuantumClassifierWithDefaultRiemannianPipeline`, qui permet de classer les ondes cérébrales en deux catégories, en combinant les avantages de l’informatique quantique avec l’analyse des signaux cérébraux pour créer un modèle de classification robuste et performant.

Concrètement, pyRiemann analyse les signaux (par exemple des électroencéphalogrammes ou des électromyogrammes) à travers plusieurs étapes : la normalisation des données, la sélection des caractéristiques pertinentes et la réduction de dimension. Qiskit est ensuite utilisé pour construire le modèle de classification : il crée des instructions pour manipuler les « qubits », les unités de base de l’ordinateur quantique, qui stockent les caractéristiques des signaux et permettent d’effectuer des calculs plus rapidement qu’un ordinateur classique. Le modèle est ensuite entraîné, puis exécuté sur un ordinateur quantique ou sur un simulateur Qiskit.

C. Justification du choix de l’algorithme

Il semblerait que l’informatique quantique offre des avantages par rapport à l’informatique classique, notamment en termes de vitesse de calcul et de résultats. Cependant, l’efficacité dépend des données utilisées, de leur qualité et de la manière dont elles sont encodées en états quantiques. Le `QuantumSVM` est tout simplement un `SVC` avec noyau quantique.

IV. Les méthodes utilisées dans le projet Titanic

A. La régression linéaire

Nous avons initialement utilisé la régression linéaire, une technique d’analyse de données qui permet de prédire la valeur de données inconnues en utilisant une autre valeur de données que l’on connaît déjà. Elle établit une relation entre une variable indépendante (x) et une variable dépendante (y), en cherchant la droite qui représente le mieux cette relation. Elle est très utile pour établir des prévisions, et donc très adaptée aux prévisions de taux de survie des passagers du Titanic.

B. Le SVR (Support Vector Regression) avec noyau RBF (Radial Basis Function)

Nous avons également utilisé le SVR, une méthode de machine learning qui, comme la régression linéaire, permet de prédire des valeurs inconnues à l'aide de valeurs connues. Cependant, au lieu de tracer une ligne droite, le SVR fonctionne en utilisant des vecteurs de support : la technique du « noyau » permet de transformer les données pour les rendre linéairement séparables et ainsi de modéliser des relations non linéaires entre les variables. Le SVR peut donc prendre en compte des relations entre x et y qui ne peuvent pas être capturées par une simple ligne droite ; c'est un modèle plus adapté lorsque les relations entre les variables sont plus complexes.

C. Méthode

Nos données sont d'abord transformées dans un état quantique avec une `ZZFeatureMap`. Pour ce travail, le nombre de répétitions a été fixé à 2, et 1024 « shots » ont été utilisés. Un shot est le nombre de fois où le circuit quantique est exécuté pour la classification : en effet, dans un ordinateur quantique, le fait de mesurer un résultat entraîne une perte d'information, et le résultat est donc en partie « erroné ». Afin de supprimer ce biais, une manière de procéder est d'exécuter l'algorithme plusieurs fois.

Ne disposant pas d'un vrai ordinateur quantique, nous avons émulé l'ordinateur avec `QasmSimulator`, qui est à l'ordinateur quantique ce que l'assembleur est à l'ordinateur classique.

D. Résultats

Les résultats obtenus pour chaque méthode sont résumés en *Table 1*.

Méthode	Résultat
Régression linéaire	0.8044
SVC	0.5977
SVC avec noyau RBF	0.5175
QuanticSVM	0.7707

Nous avons commencé par la régression linéaire, qui nous a donné un score d'accuracy de 0.8044. Puis nous avons mis en place un SVC de scikit-learn, avec lequel nous avons obtenu un score de 0.5977. Après le SVC, nous avons testé un modèle SVC avec noyau RBF, avec lequel nous avons obtenu un score de 0.5175 : ce score est inférieur au SVC, on peut donc penser que les modèles linéaires marchent mieux car le problème est ici linéaire.

Enfin, nous avons testé le QuanticSVM de la librairie pyRiemann-qiskit ; cependant le temps de chargement du modèle est beaucoup plus long que le SVC. Après deux jours de chargement, nous avons obtenu un score de 0.7707. Le modèle QuanticSVM nous donne donc un score supérieur au SVC de scikit-learn, et même au modèle SVC avec noyau RBF.

V. Conclusion

Le QuanticSVM est donc une alternative au SVC, au SVC avec noyau RBF et à la régression linéaire. Cependant, on est limité par le temps : le QuanticSVM est plus adapté aux ordinateurs quantiques, et son temps de chargement est beaucoup trop long par rapport aux autres modèles sur un PC lambda.

À l’heure actuelle, seule une poignée d’entreprises possède des ordinateurs quantiques (Google, IBM, Microsoft...). Pourtant, certaines entreprises comme IBM, avec IBM Quantum Experience, proposent l’accès à leurs ordinateurs quantiques via le cloud, et donc accessible à n’importe quelle entreprise, ou même à un usage individuel. Avec l’informatique quantique, on pourra donc réduire le temps de calcul tout en améliorant nos résultats.

Références

- Andreev, A., G. Cattani, S. Chevallier, and Q. Barthélemy. 2023. ‘pyRiemann-qiskit: A Sandbox for Quantum Classification Experiments with Riemannian Geometry’. *Research Ideas and Outcomes* 9 (March). <https://doi.org/10.3897/ri.o.9.e101006>.
- Congedo, M., A. Barachant, and R. Bhatia. 2017. ‘Fixed Point Algorithms for Estimating Power Means of Positive Definite Matrices’. *IEEE Transactions on Signal Processing* 65 (9): 2211–2220. <https://doi.org/10.1109/TSP.2017.2649483>.
- Havlíček, V., A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta. 2019. ‘Supervised Learning with Quantum-Enhanced Feature Spaces’. *Nature* 567 (7747): 209–212. <https://doi.org/10.1038/s41586-019-0980-2>.
- Liu, Y., S. Arunachalam, and K. Temme. 2021. ‘A Rigorous and Robust Quantum Speed-up in Supervised Machine Learning’. *Nature Physics* 17: 1013–1017.
- ‘Sklearn wrapper around QSVM and VQC · Issue #240 · qiskit-community/qiskit-machine-learning’. GitHub. Accessed 06/2024. <https://github.com/qiskit-community/qiskit-machine-learning/issues/240>.
- ‘pyRiemann/pyRiemann-qiskit: A Library for Machine Learning and Quantum Programming Based on pyRiemann and Qiskit Projects’. GitHub. Accessed 06/2024. <https://github.com/pyRiemann/pyRiemann-qiskit>.
- ‘Case-Based and Quantum Classification for ERP-Based Brain–Computer Interfaces’. *Brain Sciences*. Accessed 06/2024. <https://www.mdpi.com/journal/brain>

sci.

Jumpstart Reinforcement Learning with PPO

Fabien Chopin

Le-Point-Technique, 04/2024

abstract: Ce projet étudie si la méthode Jump-Start Reinforcement Learning (JSRL), qui combine une guide-policy pré-entraînée et une exploration-policy, permet d'accélérer l'apprentissage d'un agent PPO dans un environnement complexe, en comparaison avec un entraînement classique « from scratch ». Les entraînements sont réalisés dans l'environnement Point Maze de Gymnasium, avec des durées de pré-entraînement variables.

keywords: Reinforcement Learning, Proximal Policy Optimization, Jump-Start Reinforcement Learning, Gymnasium, Point Maze

1. Introduction

1.1. Le Reinforcement Learning

Pour ce projet, j'ai décidé d'explorer une partie fascinante du machine learning : le Reinforcement Learning (RL). Le principe du RL est qu'un agent (un algorithme), dans un état S , va apprendre en réalisant des actions A dans un environnement et en recevant des récompenses R selon l'impact de ces actions (*Figure 1*). Ce type de méthode n'a pas besoin d'un dataset et n'a pas besoin d'être supervisée : l'agent apprend simplement à travers les récompenses qu'il obtient.

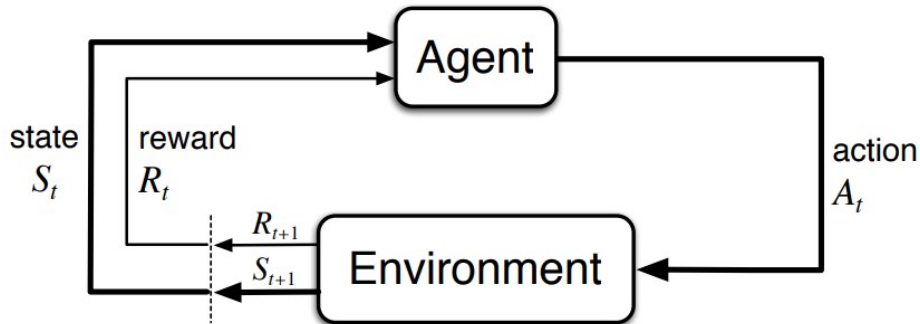


Figure 1: Boucle d'action entre un agent et un environnement

L'objectif fondamental de l'agent dans le contexte du Reinforcement Learning est de maximiser la somme cumulative des récompenses qu'il reçoit. Une caractéristique essentielle de cette approche est que l'agent prend ses décisions en fonction de l'état actuel de l'environnement, sans nécessiter la connaissance de tous les états précédents et des actions associées.

L'information transmise à l'agent depuis l'environnement peut être complète ou partielle, tandis que les actions à disposition de l'agent peuvent être finies ou infinies. De plus, les tâches d'apprentissage de l'agent peuvent être divisées en deux catégories : épisodiques, où l'agent opère dans l'environnement entre un instant initial et un instant final, et continues, où il n'y a pas de limite de temps pour l'évolution de l'agent.

Un défi majeur du Reinforcement Learning réside dans le maintien d'un équilibre entre l'exploration de nouvelles actions et l'exploitation des connaissances déjà acquises, dans le but de maximiser les récompenses à chaque étape. Si l'agent privilégie uniquement l'exploitation des connaissances existantes, il risque de ne jamais découvrir des actions potentiellement plus bénéfiques. En revanche, en se concentrant exclusivement sur l'exploration, il peut négliger de capitaliser sur les opportunités prometteuses déjà identifiées. Trouver le juste milieu entre ces deux aspects est donc crucial pour le succès de l'agent en apprentissage par renforcement.

1.2. Méthodes Policy-Based versus Value-Based

L'agent réalise des actions en suivant une stratégie appelée policy : une fonction qui, pour un état d'environnement donné, décrit l'action à prendre par l'agent. L'objectif de l'entraînement en Reinforcement Learning est de trouver une policy optimale pour maximiser les récompenses obtenues par l'agent. Pour ce faire, deux approches principales sont couramment utilisées : la méthode Policy-Based et la méthode Value-Based.

La première consiste à apprendre directement la fonction policy : à chaque état de l'environnement, on associe une action, si la policy est déterministe, ou une distribution de probabilité d'actions si la policy est stochastique. La seconde consiste à apprendre une fonction qui attribue une valeur à chaque état de l'environnement, la policy consistant alors à aller vers des états d'environnement avec une valeur plus élevée que celle de l'état actuel.

La méthode Policy-Based est plus efficace dans un espace d'actions très grand, voire infini ; elle a l'avantage de pouvoir apprendre des politiques stochastiques et possède de meilleures propriétés de convergence. La méthode Value-Based est plus efficace pour trouver un optimum global, elle est plus rapide et présente une variance plus faible.

1.3. Proximal Policy Optimization

L'entraînement d'un agent est soumis à l'aléatoire de l'environnement et de la policy si elle est stochastique. Il existe alors une forte variance dans les récompenses obtenues pour un même état initial de l'environnement. Pour réduire cette variance, on peut envisager d'augmenter fortement le nombre de pas de temps d'apprentissage, mais cela allongerait le temps d'entraînement de manière importante. La méthode Proximal Policy Optimization (PPO) permet de résoudre ce problème en alliant deux idées :

- utiliser une méthode dite Actor-Critic, qui consiste à combiner les méthodes Policy-Based et Value-Based ;
- limiter l'importance du changement de la policy entre deux mises à jour de ses poids lors de l'entraînement.

1.4. Problématique

La PPO permet de réduire efficacement la variance en stabilisant les entraînements, et la méthode Jump-Start Reinforcement Learning (JSRL), qui dédie une policy à l'exploitation et une à l'exploration, permet de prendre en compte le compromis mentionné précédemment. Il reste un aspect important à prendre en compte : le temps d'entraînement. Dans ce projet, je traite donc de la problématique suivante :

La méthode JSRL permet-elle d'accélérer l'apprentissage d'un environnement complexe par un agent PPO, en utilisant un autre agent PPO pré-entraîné dans un environnement moins complexe ?

Je vais d'abord détailler la méthode utilisée pour répondre à cette problématique, puis présenter les résultats obtenus, pour finalement en discuter et conclure.

2. Méthodologie

2.1. Jump-Start Reinforcement Learning

Le JSRL est un algorithme qui utilise deux policies : une guide-policy et une exploration-policy. La guide-policy est une policy meilleure qu'une policy aléatoire. Elle est utilisée dans les premiers pas de temps d'un épisode pour atteindre de « bons » états d'environnement, avant de laisser l'exploration-policy reprendre le contrôle. Au fur et à mesure de l'entraînement, l'exploration-policy devient de plus en plus performante : la guide-policy est donc de moins en moins utilisée, pour finalement laisser un contrôle intégral à l'exploration-policy et retourner dans un cadre de Reinforcement Learning classique (*Figure 2*).

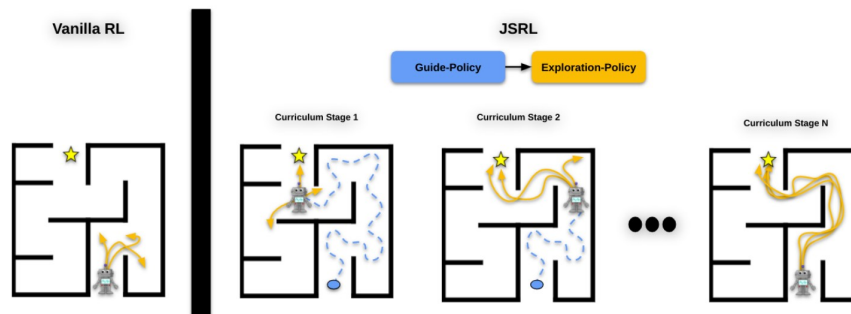


Figure 2: Illustration de l'algorithme Jump-Start Reinforcement Learning

Cette méthode possède trois hyperparamètres :

- le nombre initial de pas de temps utilisé par la guide-policy : `max_horizon` ;
- le nombre d'étapes de transfert progressif entre guide-policy et exploration-policy (N) ;
- le seuil bêta de performance souhaitée avant de passer de l'étape de transfert k à k+1.

Pour l'ensemble des entraînements concernés par le JSRL, j'ai fixé N=10 et `max_horizon`=100. Tous les 2000 pas de temps, l'agent est évalué sur 100 épisodes ; la moyenne des récompenses obtenues lors de ces 100 épisodes est conservée, et au cours de l'entraînement, le seuil bêta prend la valeur du maximum de ces moyennes.

2.2. Entraînements réalisés

Dans le projet, j'ai voulu répondre à la problématique dans l'environnement Point Maze Open-v3 de Gymnasium (Farama Foundation, n.d.), une API vers des environnements standards de Reinforcement Learning. J'ai choisi des épisodes d'une durée maximale de 500 pas de temps, avec un unique objectif : atteindre la cible. La récompense est de 0 si la cible n'est pas atteinte avant la fin des 500 pas de temps ; dans le cas contraire, l'épisode se termine avant les 500 pas et la récompense est de 1.

Ensuite, j'ai comparé deux types d'entraînement :

- **Un entraînement dit « from scratch »** : un entraînement classique de Reinforcement Learning pour un ensemble de 200 000 pas de temps, avec un agent PPO déterministe.
- **Un entraînement dit « JSRL-pretraining_steps »** : dans un premier temps, un agent PPO déterministe est entraîné dans l'environnement moins complexe Point Maze Open Diverse G-v3, où la position de départ est fixe (contrairement à Point Maze Open) quel que soit l'épisode, la position de la cible restant aléatoire. Ce pré-entraînement est réalisé pour un nombre prédéfini de pas de temps (`pretraining_steps`). Dans un second temps, la policy de cet agent est utilisée comme guide-policy dans un algorithme de JSRL au sein de l'environnement Point Maze Open-v3, l'exploration-policy étant utilisée par un agent PPO déterministe. Le nombre de pas de temps d'entraînement JSRL est lui aussi de 200 000. L'implémentation de cette deuxième phase utilise le dépôt GitHub de (Tang 2024).

Au total, 10 entraînements from scratch, 10 JSRL-25000, 10 JSRL-50000 et 10 JSRL-100000 sont réalisés.

3. Résultats

Les résultats de ces entraînements sont représentés en *Figure 3* et *Figure 4*.

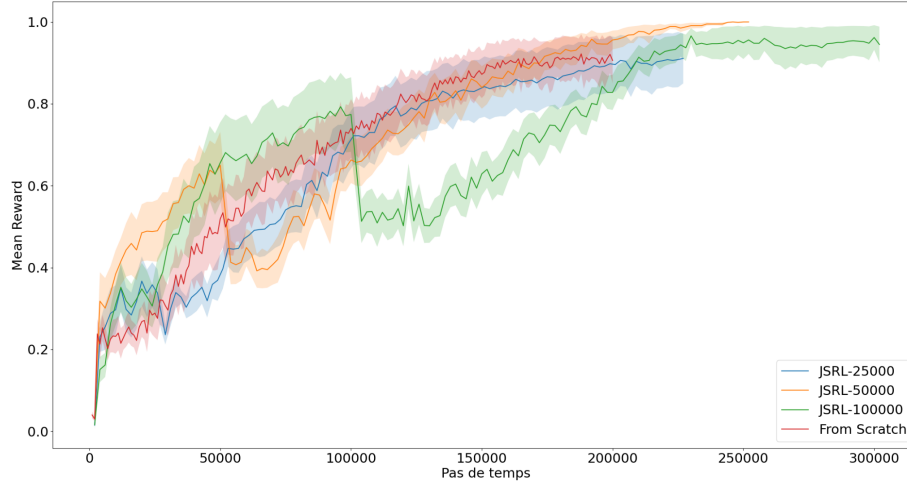


Figure 3: Récompenses cumulées moyennes des agents, pas de temps final commun

Les pas de temps négatifs correspondent aux pré-entraînements dans l'environnement moins complexe. On constate qu'en moyenne, pré-entraîner pendant 50 000 ou 100 000 pas permet d'obtenir des agents plus performants qu'un entraînement from scratch à l'issue des 200 000 pas de temps. Cependant, un pré-entraînement de 25 000 pas ne semble pas suffisant pour améliorer les performances.

Les résultats présentés en *Figure 4* montrent, à pas de temps égal, que JSRL ne permet pas d'apprendre plus rapidement qu'un entraînement from scratch, et implique même un entraînement global plus long lors de JSRL-100000.

Enfin, par souci de visibilité, les écarts types représentés de part et d'autre des moyennes ont été divisés par 4 sur les deux figures : on constate donc une grande variabilité des résultats.

4. Discussion

En ayant à disposition une policy pré-entraînée, JSRL semble accélérer l'apprentissage en environnement plus complexe. C'est un outil qui permet de transmettre efficacement des connaissances d'une policy à une autre. Au contraire, sans policy pré-entraînée, il ne semble pas efficace, en termes de temps, de se pré-entraîner à une tâche plus simple avant de s'entraîner à la tâche voulue : une décomposition de l'entraînement en tâche simple puis complexe n'est pas pertinente dans notre cas.

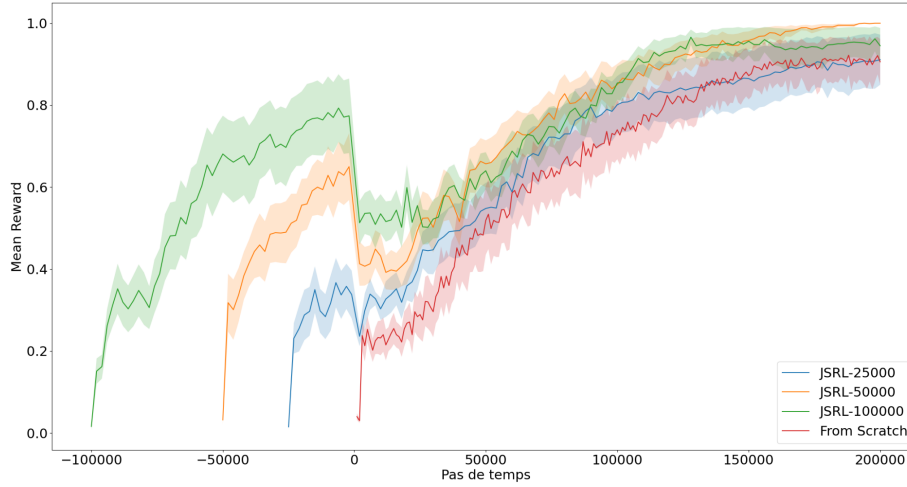


Figure 4: Récompenses cumulées moyennes des agents, pas de temps initial commun

Pour des questions de temps de projet, de capacités de calcul et de simplicité de mise en place, plusieurs limitations sont à noter :

- dans l'article de recherche de référence, l'agent utilisé pour les guide- et exploration-polices était un Implicit Q-Learning et non un PPO. Bien qu'il s'agisse d'une méthode Actor-Critic, elle est utilisée avec un pré-entraînement basé sur un dataset hors-ligne (un ensemble fini d'états, d'actions et de récompenses regroupés dans un dataset, l'agent n'ayant pas d'accès direct à l'environnement) ;
- l'environnement Point Maze Open est relativement simple en comparaison des environnements présentés dans l'article de recherche : il s'agit d'un simple carré, sans obstacle, comme le montre la *Figure 5* ;
- finalement, aucune optimisation des hyperparamètres n'a été faite. Le `max_horizon` a été choisi proche du nombre de pas de temps moyen nécessaire à un agent entraîné from scratch pour résoudre une tâche dans Point Maze Open-v3. Le paramètre `N` a été arbitrairement fixé à 10 ; il pourrait avoir un impact sur la rapidité de transfert de connaissances. Enfin, le seuil de performance pourrait influencer la vitesse et la qualité de ce transfert.

5. Conclusion

La méthode JSRL ne semble pas accélérer un entraînement pour une toute nouvelle tâche dans un environnement, mais elle semble permettre de valoriser

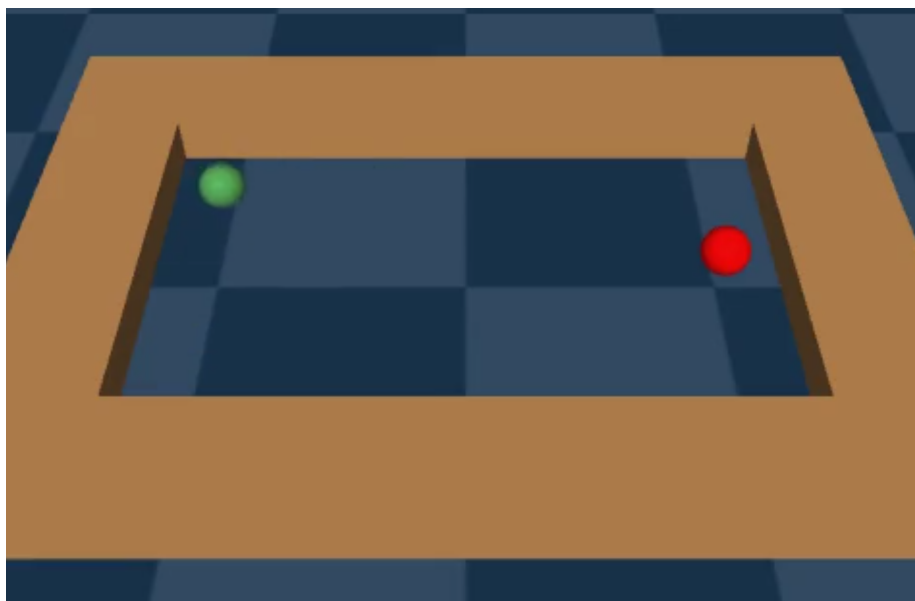


Figure 5: Exemple d'état initial d'un épisode de Point Maze Open-v3

efficacement une policy pré-existante pour la généraliser à un environnement ou une tâche plus complexe que lors de sa création.

Références

- Sutton, R. S., and A. G. Barto. 2020. 'Reinforcement Learning: An Introduction'. <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>.
- Uchendu, I., T. Xiao, Y. Lu, B. Zhu, M. Yan, J. Simon, M. Bennice, et al. 2023. 'Jump-Start Reinforcement Learning'. arXiv, July 7, 2023. Accessed April 11, 2024. <http://arxiv.org/abs/2204.02372>.
- Farama Foundation. n.d. 'Gymnasium-Robotics Documentation'. Accessed April 11, 2024. https://robotics.farama.org/envs/maze/point_maze.html.
- Tang, S. 2024. 'jumpstart-rl'. April 10, 2024. Accessed April 11, 2024. <https://github.com/steventango/jumpstart-rl>.

Mentions Légales

Titre de la revue : Le-Point-Technique

Éditeur de la publication : Grégoire Cattan, Montpellier, France

ISSN : 2826-5726